TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS • CAPÍTULO 5

Python e PostgreSQL

Conectar um programa ao banco na nuvem, com segurança, e construir um CRUD de verdade

```
banco = Banco()
linhas = banco.consultar(
    "SELECT nome_produto FROM produtos WHERE preco < %s",
    (100,),
)
```

PROFESSOR
Tarik Ponciano

Roteiro

01 **Python conversa com o banco**
O `.env`, o psycopg e o `setup_rh.py`: apagar, criar e preencher o banco

02 **O primeiro sistema**
Um menu que consulta e cadastra, com a conexão em cada opção e os valores por `%s`

03 **A classe Banco e o CRUD**
A conexão num lugar só e o CRUD completo do Sistema RH

04 **Prática guiada: Loja Conecta**
O CRUD de produtos e o fluxo de venda: o pedido e os itens gravados juntos

★ **Atividade final**
Um tema sorteado para cada um: do briefing ao sistema funcionando

NESTE MATERIAL

Exemplo explicado
Um caso resolvido passo a passo.

Prática
Individual e digital: o que for feito no Sistema RH, você repete no seu sistema do **Cinema Estrela**.

Terminal
Os programas foram testados: a saída mostrada é a que aparece na tela.

ONDE ESTAMOS

O que muda neste capítulo

NOS CAPÍTULOS 3 E 4

Quem falava com o banco era você

Cada comando era digitado na Query Tool do pgAdmin, conectado à Aiven.

NESTE CAPÍTULO • SEÇÃO 01

Um **programa Python** conecta na Aiven e monta o banco, com a senha guardada fora do código.

NA UC 1

O CRUD guardava tudo numa lista

```
funcionarios.append(novo_funcionario)
```

E salvava num `dados.json`, só na sua máquina.

NESTE CAPÍTULO • SEÇÕES 02 E 03

Um menu com os dados no **PostgreSQL**: primeiro simples, depois com a classe **Banco** e o CRUD completo.

NO CAPÍTULO 4

A Loja Conecta respondia ao JOIN

O banco estava pronto, mas só para consultas no pgAdmin.

NESTE CAPÍTULO • SEÇÃO 04

A loja ganha um **CRUD de produtos** e um **fluxo de venda**: o pedido e os itens gravados juntos.

Em cada etapa, o que for feito no Sistema RH você repete no seu sistema do **Cinema Estrela**. No fim, cada um sorteia um tema e vai do briefing ao sistema funcionando.

SEÇÃO 01

Python conversa com o banco

01

O arquivo .env, o psycopg e o primeiro script: apagar, criar e preencher o banco.

01 • CONEXÃO

Do pgAdmin ao programa

Num sistema de verdade, ninguém digita SQL. O usuário escolhe uma opção no menu, e o programa monta o comando, envia ao banco e mostra o resultado.

Usuário
escolhe no menu e digita os dados

→

Programa Python
monta o SQL com os dados digitados

→

psycopg
leva o comando pela internet e traz a resposta

→

PostgreSQL na Aiven
confere as regras e guarda os dados

Os dados ficam

Fechou o programa, desligou o PC: os dados continuam no servidor. O `dados.json` da UC 1 morava só na sua máquina.

Vários lugares, um banco

O mesmo banco atende o laboratório, a sua casa e, um dia, o app do celular.

As regras protegem

As FKs e os CHECKs do Capítulo 4 recusam o dado errado, venha do pgAdmin ou do Python.

psycopg é a biblioteca que faz a ponte entre o Python e o PostgreSQL: o programa fala Python, o banco fala SQL, e o psycopg leva e traz.

O que instalar

No terminal do VS Code, uma vez só por computador:

```
TERMINAL
> pip install "psycopg[binary]" python-dotenv
...
Successfully installed psycopg-3.3.6 psycopg-binary-3.3.6
python-dotenv-1.1.1
```

PACOTE	PARA QUÊ
`psycopg`	A ponte entre o Python e o PostgreSQL. No código: `import psycopg`.
`[binary]`	Vem pronto para o Windows: não precisa instalar mais nada.
`python-dotenv`	Lê o arquivo `.env`, onde fica a senha. No código: `from dotenv import load_dotenv`.

CONFERIR

```
TERMINAL
> python -c "import psycopg;
print(psycopg.__version__)"
3.3.6
```

Apareceu um número? Está instalado. `ModuleNotFoundError`? O pip instalou num Python diferente do que o VS Code usa: rode o pip no terminal do próprio VS Code.

Na internet você vai ver muito **psycopg2**, a versão anterior. A ideia é a mesma; neste capítulo usamos a versão 3: `import psycopg`.

01 • CONEXÃO • AIVEN

Os dados da conexão estão na Aiven

Os mesmos do Capítulo 3, quando o pgAdmin conectou na Aiven. Em **console.aiven.io** → seu serviço → **Overview** → **Connection information**.

NA AIVEN	NO PGADMIN (CAPÍTULO 3)	NO PYTHON	EXEMPLO
Host	Host name/address	`host`	`pg-escola-turma.a.aivencloud.com`
Port	Port	`port`	`12691` (a do seu serviço)
Database name	Maintenance database	`dbname`	`rh` (o banco que você criou)
User	Username	`user`	`avnadmin`
Password	Password	`password`	a do console da Aiven
SSL mode	SSL mode	`sslmode`	`require`

require: a conversa com a Aiven vai criptografada. Sem ele, a Aiven recusa a conexão.

A senha é sua. Quem tem esses seis dados entra no seu banco: lê, altera e apaga tudo.

Por que não escrever a senha no código

ASSIM NÃO

setup_rh.py

```
conexao = psycopg.connect(
    host="pg-escola-turma.a.aivencloud.com",
    port=12691,
    dbname="rh",
    user="avnadmin",
    password="AVNS_x8Kq2pW7rT",    # a senha no código
    sslmode="require",
)
```

`git push` → `github.com/você/projeto` → `qualquer pessoa lê`

- O código vai para o **GitHub**. A senha não pode ir junto.
- Existem robôs que varrem o GitHub atrás de senhas e chaves publicadas.
- Cada aluno tem a sua senha: com ela no código, o colega precisa editar o programa para rodar.
- Trocou a senha na Aiven? Com a senha num arquivo à parte, muda um arquivo só, e o código fica igual.

A solução: o arquivo .env

Os dados da conexão ficam num arquivo separado, que **não** vai para o GitHub. O código só lê esse arquivo.

⚠ Senha que foi para o GitHub é senha vazada: mesmo apagando depois, ela fica no histórico dos commits. Troque a senha no console da Aiven.

O arquivo .env

.env

```
# Conexão com a Aiven (não vai para o GitHub)
DB_HOST=pg-escola-turma.a.aivencloud.com
DB_PORT=12691
DB_NAME=rh
DB_USER=avnadmin
DB_PASSWORD=AVNS_sua_senha_aqui
DB_SSL=require
```

PARECE UM DICIONÁRIO

```
{"DB_HOST": "pg-escola-turma.a.aivencloud.com", "DB_PORT": "12691", ...}
```

Uma chave, um valor. E todo valor chega no Python como **texto**: até a porta é `"12691"`.

- Uma informação por linha: `NOME=valor`.
- Sem espaço em volta do `=` e sem aspas: o formato mais simples, que toda ferramenta aceita.
- Nomes em MAIÚSCULAS, por costume. O `DB_` lembra que é do banco.
- `#` no começo da linha: comentário.
- O nome do arquivo é só `.env`: nada antes do ponto, nada de `.txt` depois.
- Fica na **mesma pasta** do programa.

Crie pelo VS Code: **New File** → `.env`. O Bloco de Notas costuma salvar como `.env.txt`.

O .env fica fora do GitHub

```
rh/
├── .env            sua senha · não vai para o GitHub
├── .env.exemplo    o modelo, sem senha · vai
├── .gitignore      a lista do que o git ignora
├── setup_rh.py
└── testar_conexao.py
```

.gitignore

```
.env
```

.env.exemplo

```
DB_HOST=seu-servico.a.aivencloud.com
DB_PORT=12345
DB_NAME=rh
DB_USER=avnadmin
DB_PASSWORD=cole_a_senha_aqui
DB_SSL=require
```

1. Crie o `.gitignore` com a linha `.env` **antes** do primeiro commit. O git passa a fingir que o arquivo não existe.
2. Suba o `.env.exemplo`: quem baixar o projeto copia, renomeia para `.env` e coloca os próprios dados.
3. Antes de cada commit, confira o `git status`: o `.env` não pode aparecer na lista.

TERMINAL

```
> git status
Untracked files:
        .env.exemplo
        .gitignore
        setup_rh.py
(o .env não aparece: está sendo ignorado)
```

No repositório da disciplina, o `.gitignore` já tem a linha `.env`.

Lendo o .env no programa

```python
import os
from dotenv import load_dotenv

load_dotenv()                       # procura o .env e carrega os valores

servidor = os.getenv("DB_HOST")     # 'pg-escola-turma.a.aivencloud.com'
porta = os.getenv("DB_PORT")        # '12691'  (texto)
senha = os.getenv("DB_SENHA")       # None: esse nome não existe no .env
```

Compare com o dicionário: `os.getenv("DB_HOST")` funciona como `dados.get("DB_HOST")`. Chave que não existe devolve `None`, sem erro.

COMANDO	O QUE FAZ
`import os`	Biblioteca do próprio Python para falar com o sistema operacional.
`load_dotenv()`	Procura o `.env` na pasta do programa (e, se não achar, nas pastas de cima) e carrega cada linha no **ambiente**.
`os.getenv("NOME")`	Lê um valor do ambiente. Sempre texto, ou `None`.

Ambiente são variáveis que o sistema operacional entrega a cada programa. O `load_dotenv` copia o `.env` para lá, e o `os.getenv` lê de lá.

As peças do psycopg

Cada coisa que você fazia no pgAdmin tem uma linha equivalente no Python.

NO PGADMIN	NO PYTHON, COM PSYCOPG	O QUE É
Conectar no servidor, com a senha	`conexaoDB = psycopg.connect(host=..., ...)`	A **conexão**: a ligação com o servidor.
Abrir a Query Tool	`cursor = conexaoDB.cursor()`	O **cursor**: quem roda os comandos e guarda a resposta.
Escrever o SQL e apertar F5	`cursor.execute("SELECT ...")`	O SQL vai como texto, igual ao da Query Tool.
Ver a grade do resultado	`cursor.fetchall()`	A lista com todas as linhas da resposta.
Nada: salva sozinho a cada comando	`conexaoDB.commit()` • `conexaoDB.rollback()`	Grava de vez • desfaz o que ficou pendente.
Fechar a aba e desconectar	`cursor.close()` • `conexaoDB.close()`	Libera o cursor e a conexão no servidor.

A ordem é sempre essa: **conectar** → **cursor** → **execute** → **(fetchall ou commit)** → **fechar**. O resto do capítulo é variação desse molde.

01 • A SITUAÇÃO DA EXPOSIÇÃO

O Sistema RH ganha um banco

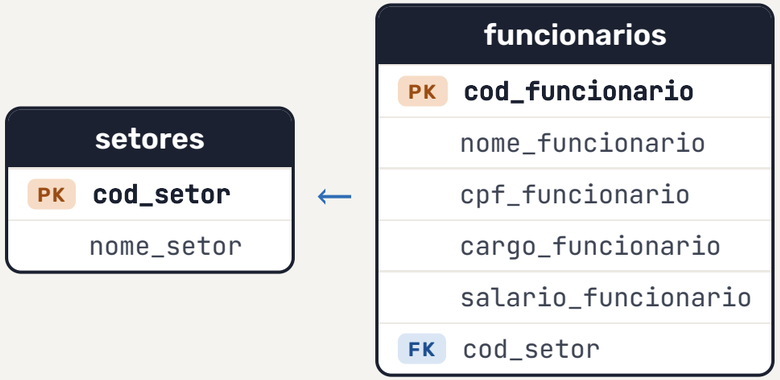
O Sistema de Funcionários da UC 1 guardava cada funcionário num dicionário. Agora cada dicionário vira uma **linha**, e cada chave, uma **coluna**.

NA UC 1

```
novo_funcionario = {
    "nome": nome,
    "cpf": cpf,
    "cargo": cargo,
    "salario": salario
}
funcionarios.append(novo_funcionario)
```

REGRA	NA UC 1	NO BANCO
CPF não se repete	`cpf_exist(cpf)`	`UNIQUE`
CPF com 11 dígitos	`len(cpf) != 11`	`ck_cpf_tamanho`
Salário maior que zero	`salario <= 0`	`ck_salario_positivo`
Setor existe	não havia	`fk_func_setor`
Código do funcionário	a posição na lista	`GENERATED ALWAYS`

NESTE CAPÍTULO • BANCO RH



As regras da UC 1, agora também no banco: valem para qualquer programa que use o banco rh.

E se der errado?

Um programa que só conecta, sem `try`, com a senha errada no `.env`:

TERMINAL

```
> python conectar.py
Traceback (most recent call last):
  File "D:\projetos\rh\conectar.py", line 7, in <module>
    conexaoDB = psycopg.connect(
  File "...\site-packages\psycopg\connection.py", line 134, in connect
    raise new_ex.with_traceback(None)
psycopg.OperationalError: connection failed: connection to server at "34.39.x.x", port 12691 failed:
FATAL:  password authentication failed for user "avnadmin"
```

O programa morre

Nenhuma linha depois do erro roda. Num menu, o usuário perde o programa inteiro por causa de uma opção.

A mensagem assusta

Quem usa o sistema não sabe o que é *Traceback*. Bastava dizer: “não foi possível conectar”.

A conexão pode ficar aberta

Se o erro acontece **depois** de conectar, o `close()` nunca roda. A conexão fica presa no servidor, segurando as tabelas.

Na UC 1, o `try/except ValueError` salvou o `int(input())`. Com banco, a ideia é a mesma, com uma peça nova: o **finally**.

setup_rh.py: try, except e finally

setup_rh.py

```python
# os imports e o load_dotenv(), como no slide do .env
conexaoDB = None
cursor = None
try:
    # 1. Criação e configuração da conexão e do cursor
    conexaoDB = psycopg.connect(
        host=os.getenv("DB_HOST"),
        port=os.getenv("DB_PORT"),
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        sslmode=os.getenv("DB_SSL")
    )
    cursor = conexaoDB.cursor()
    # 2. SQL que será executado (próximos slides)
    # Grava tudo de uma vez
    conexaoDB.commit()
except Exception as erro:
    print("ACONTECEU UM ERRO:", erro)
    # Desfaz o que estava pendente
    if (conexaoDB is not None):
        conexaoDB.rollback()
finally:
    # 3. Finalização do processo: fechar o cursor e a conexão
    if (cursor is not None):
        cursor.close()
    if (conexaoDB is not None):
        conexaoDB.close()
```

PARTE	QUANDO RODA E O QUE FAZ
`= None`	Antes de tudo: ainda não há conexão nem cursor. O `finally` precisa saber disso.
`try`	O caminho normal: conectar, rodar o SQL e o `commit()`. Para no primeiro erro.
`except Exception`	Só se der erro, **qualquer** erro: mostra a mensagem e o `rollback()` desfaz o que ficou pendente.
`finally`	**Sempre**, no fim, com erro ou sem erro: fecha o cursor e a conexão.

Na UC 1, o `try/except ValueError` salvou o `int(input())`. A peça nova é o **finally**: o lugar de fechar o que foi aberto.

O mesmo script, três finais

TUDO CERTO

`try`: conecta, apaga, cria, insere

↓

`commit()`: grava tudo

↓

o **`except`** não roda

↓

`finally`: fecha o cursor e a conexão

NÃO CONECTOU

`try`: o **`connect`** falha na 1ª linha

↓

`except`: mostra o erro; **`conexaoDB`** ainda é **`None`**, não há o que desfazer

↓

`finally`: nada foi aberto, nada a fechar

UM COMANDO FALHOU

`try`: conecta, apaga… e o CREATE falha

↓

`except`: mostra o erro e o **`rollback()`** desfaz até o DROP

↓

`finally`: fecha o cursor e a conexão

Por que `conexaoDB = None` e `cursor = None` antes do `try`? No caminho do meio, o `connect` falhou e as variáveis nunca receberiam valor. Sem essas linhas, o `finally` quebraria com `NameError`.

PENSE E RESPONDA No caminho da direita, o DROP já tinha rodado quando o CREATE falhou. Depois do rollback, as tabelas antigas existem ou não?

setup_rh.py: apagar e criar

```
# 2. SQL que será executado: apagar (filhos antes dos pais)...
cursor.execute("DROP TABLE IF EXISTS funcionarios;")
cursor.execute("DROP TABLE IF EXISTS setores;")
print("REMOÇÃO DE TABELAS ANTIGAS CONCLUÍDA")

# ... criar (pais antes dos filhos) ...
cursor.execute('''
    CREATE TABLE setores(
        cod_setor INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        nome_setor VARCHAR(255) NOT NULL UNIQUE
    );
''')
cursor.execute('''
    CREATE TABLE funcionarios(
        cod_funcionario INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        nome_funcionario VARCHAR(255) NOT NULL,
        cpf_funcionario CHAR(11) NOT NULL UNIQUE,
        cargo_funcionario VARCHAR(255) NOT NULL,
        salario_funcionario NUMERIC(10,2) NOT NULL,
        cod_setor INTEGER NOT NULL,
        CONSTRAINT ck_cpf_tamanho CHECK (LENGTH(cpf_funcionario) = 11),
        CONSTRAINT ck_salario_positivo CHECK (salario_funcionario > 0),
        CONSTRAINT fk_func_setor FOREIGN KEY (cod_setor) REFERENCES setores (cod_setor)
    );
''')
print("TABELAS CRIADAS COM SUCESSO!")
```

- Cada comando SQL vira um `cursor.execute(...)`, um depois do outro, no lugar do `# 2` do molde.
- O SQL vai entre aspas: para o Python é só um **texto**. As aspas triplas `'''` deixam escrever em várias linhas.
- A ordem é a do Capítulo 4: apagar **filhos antes dos pais**, criar **pais antes dos filhos**.
- O `IF EXISTS` deixa o script rodar de novo.
- Os `print` mostram até onde o script chegou.

Dica: teste o `CREATE` no pgAdmin. Funcionou lá? Cole entre as aspas.

setup_rh.py: preencher e gravar

```
# ... e inserir os dados (pais antes dos filhos)
cursor.execute('''
    INSERT INTO setores (nome_setor)
    VALUES ('Administrativo'), ('Comercial'), ('Tecnologia');
''')
cursor.execute('''
    INSERT INTO funcionarios
        (nome_funcionario, cpf_funcionario, cargo_funcionario, salario_funcionario, cod_setor)
    VALUES ('Ana Souza', '12345678901', 'Analista de RH', 4200.00, 1),
           ('Bruno Lima', '23456789012', 'Vendedor', 3100.00, 2),
           ('Carla Mendes', '34567890123', 'Desenvolvedora', 6800.00, 3),
           ('Diego Rocha', '45678901234', 'Suporte técnico', 3500.00, 3),
           ('Elisa Martins', '56789012345', 'Gerente comercial', 7500.00, 2);
''')
print("DADOS INSERIDOS.")

# Grava tudo de uma vez
conexaoDB.commit()
```

- Os códigos não aparecem no INSERT: o `GENERATED` numera 1, 2, 3… na ordem das linhas.
- Por isso o setor 1 é Administrativo, o 2 é Comercial e o 3 é Tecnologia.
- `commit()` grava de vez tudo o que o script fez. Sem ele, nada fica (próximos slides).

EXEMPLO EXPLICADO • RODANDO O SETUP

O banco rh montado pelo Python

TERMINAL

```
> python setup_rh.py
REMOÇÃO DE TABELAS ANTIGAS CONCLUÍDA
TABELAS CRIADAS COM SUCESSO!
DADOS INSERIDOS.
```

- Rode de novo: o resultado é o mesmo. O DROP apaga, o CREATE recria e o INSERT preenche: o banco **sempre começa do zero**.
- Um colega baixa o projeto, cria o próprio `.env` e roda o mesmo script: tem o mesmo banco, na Aiven dele.

CONFIRA NO PGADMIN

Botão direito em **Tables** → **Refresh**. Na Query Tool do banco rh:

```
SELECT * FROM funcionarios;
```

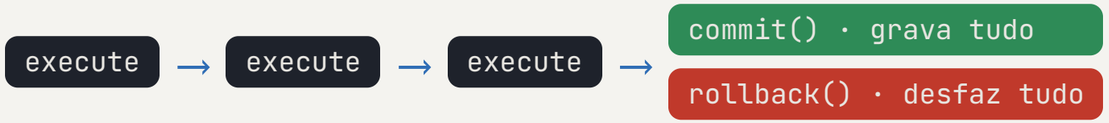
cod_funcionario	nome_funcionario	cargo_funcionario	salario_funcionario	cod_setor
1	Ana Souza	Analista de RH	4200.00	1
2	Bruno Lima	Vendedor	3100.00	2
3	Carla Mendes	Desenvolvedora	6800.00	3
4	Diego Rocha	Suporte técnico	3500.00	3
5	Elisa Martins	Gerente comercial	7500.00	2

5 linhas · sem a coluna cpf_funcionario, para caber

01 • TRANSAÇÃO

Sem commit, nada fica gravado

No primeiro `execute`, o psycopg abre uma **transação**: tudo o que vem depois fica pendente, esperando uma decisão.



- `close()` sem `commit()` desfaz o que estava pendente.
- No pgAdmin você nunca precisou: a Query Tool vem com o **auto-commit** ligado, e cada comando é gravado na hora.
- No PostgreSQL até o `DROP` e o `CREATE` entram na transação e podem ser desfeitos.

TESTE: O MESMO SCRIPT, SEM A LINHA DO COMMIT

```
TERMINAL
> python setup_rh.py
REMOÇÃO DE TABELAS ANTIGAS CONCLUÍDA
TABELAS CRIADAS COM SUCESSO!
DADOS INSERIDOS.
```

O programa parece ter dado certo, mas no pgAdmin, depois do Refresh, **não há nenhuma tabela**. Nenhum erro, nenhum aviso: o erro mais traiçoeiro do capítulo.

Tudo ou nada: é isso que a Seção 04 usa para gravar um pedido com todos os itens, ou não gravar nada.

EXEMPLO EXPLICADO • TRY, EXCEPT E FINALLY

Três execuções do mesmo script

1 • TUDO CERTO

TERMINAL

```
> python setup_rh.py
REMOÇÃO DE TABELAS ANTIGAS CONCLUÍDA
TABELAS CRIADAS COM SUCESSO!
DADOS INSERIDOS.
```

O `try` foi até o `commit()`, e o `finally` fechou tudo.

2 • SENHA ERRADA NO .ENV

TERMINAL

```
> python setup_rh.py
ACONTECEU UM ERRO: connection failed:
connection to server at "34.39.x.x",
port 12691 failed: FATAL:  password
authentication failed for user
"avnadmin"
```

O `connect` falhou: nenhum `print` do `try` apareceu, e não havia nada a desfazer.

3 • CREAT NO LUGAR DE CREATE

TERMINAL

```
> python setup_rh.py
REMOÇÃO DE TABELAS ANTIGAS CONCLUÍDA
ACONTECEU UM ERRO: syntax error at or
near "CREAT"
LINE 2:         CREAT TABLE
funcionarios(
                ^
```

O DROP rodou, o CREATE falhou. O `rollback()` desfez até o DROP: **as tabelas antigas continuam lá**, com os 5 funcionários.

OUTRAS MENSAGENS DE CONEXÃO

```
ACONTECEU UM ERRO: connection failed: ... FATAL:  database "rhh" does not exist
```

o `DB_NAME` não bate com o banco criado na Aiven

```
ACONTECEU UM ERRO: failed to resolve host 'pg-escola-turmaa.a.aivencloud.com': [Errno 11001] getaddrinfo failed
```

o `DB_HOST` tem erro de digitação, ou o computador está sem internet

EXEMPLO EXPLICADO • .ENV • ERROS COMUNS

Quando o .env não é lido

SITUAÇÃO	O QUE APARECE NO SETUP_RH.PY	COMO RESOLVER
O arquivo ficou `.env.txt`, ou está em outra pasta	`ACONTECEU UM ERRO: connection failed: connection to server at "localhost" (::1), port 5432 failed: fe_sendauth: no password supplied`	Sem o `.env`, todo `os.getenv` deu `None`, e o psycopg tentou um PostgreSQL **no próprio computador**. Renomeie pelo VS Code e deixe o arquivo na pasta do programa.
Faltou o `load_dotenv()`	`ACONTECEU UM ERRO: ... fe_sendauth: no password supplied`	O mesmo caso: nada foi lido. Sem PostgreSQL instalado, aparece *Connection refused*.
`DB_PASS` no .env, `DB_PASSWORD` no código	`ACONTECEU UM ERRO: ... fe_sendauth: no password supplied`	O nome precisa ser **igual** nos dois lugares, letra por letra.

Na dúvida, imprima o que foi lido: `print(os.getenv("DB_HOST"))`. Saiu `None`? O problema está no `.env`, não no banco. Só não imprima a senha.

Exception pega todos os erros

Neste capítulo, o `except` é sempre o **genérico**: `except Exception as erro`. Ele pega qualquer erro, e o `print` mostra a mensagem do banco, que diz o que aconteceu:

A MENSAGEM FALA DE…	O QUE ACONTECEU
`connection failed`	Não conectou: senha, host, banco ou internet.
`syntax error` • `does not exist`	O SQL está escrito errado, ou a tabela ou coluna não existe.
`duplicate key` • `violates … constraint`	Uma regra do Capítulo 4 recusou: UNIQUE, FK, CHECK, NOT NULL.

PARA SABER: CADA ERRO TEM UM TIPO

O psycopg dá um tipo para cada caso: `OperationalError` (conexão), `errors.UniqueViolation`, `errors.ForeignKeyViolation`, `errors.CheckViolation` … Um `except` para cada tipo permite uma mensagem diferente para cada regra:

```
except psycopg.errors.UniqueViolation:
    print("CPF já cadastrado.")
except Exception as erro:
    print("ACONTECEU UM ERRO:", erro)
```

O específico vem antes do genérico. Exemplo completo no repositório: `rh/testar_erros.py`.

Por que fechar a conexão importa

O servidor tem limite

Cada conexão aberta ocupa memória e uma vaga no servidor. O PostgreSQL aceita um número limitado de conexões ao mesmo tempo (o padrão é 100), e o plano gratuito da Aiven é um servidor pequeno.

Transação aberta segura as tabelas

Uma conexão que fez um `DROP` ou um `UPDATE` e não terminou segura aquela tabela. No pgAdmin, o `SELECT` fica “rodando” sem resposta até a conexão fechar.

Programa que não termina

O `setup` acaba em segundos, e o sistema fecha tudo ao sair. Um **menu** fica aberto a aula inteira: cada erro sem `close()` deixa uma conexão para trás.

O finally não esquece

Ele roda com erro, sem erro e até depois de um `return`. Com o `close()` lá, não importa como o `try` terminou.

Regra do capítulo: **abriu, fecha no finally**. Na Seção 03, a classe Banco faz isso por você em toda consulta.

PRÁTICA 1 • 20 MIN • INDIVIDUAL

O setup do Cinema Estrela

O Cinema Estrela quer um banco para os filmes e as sessões. Na Aiven, crie o banco **cinema** e escreva o `setup_cinema.py` no molde do `setup_rh.py`.

TABELA • COLUNA	REGRA
`filmes.cod_filme`	gerado pelo banco (GENERATED ALWAYS)
`filmes.titulo_filme`	texto até 100, obrigatório
`filmes.duracao_filme`	inteiro, em minutos, obrigatório • maior que zero: `ck_duracao`
`filmes.classificacao_filme`	texto até 2, obrigatório • só L, 10, 12, 14, 16 ou 18: `ck_classificacao`
`sessoes.cod_sessao`	gerado pelo banco
`sessoes.cod_filme`	obrigatória • FK para filmes: `fk_sessao_filme`
`sessoes.data_sessao` • `horario_sessao`	`date` e `time`, obrigatórios
`sessoes.sala_sessao`	inteiro, obrigatório • de 1 a 5: `ck_sala`

1. Na mesma pasta do `setup_rh.py`, crie o `setup_cinema.py`. O `.env` é o mesmo: no `connect`, troque só o banco: `dbname="cinema"`.
2. DROP das duas tabelas, CREATE com as regras e INSERT dos dados do próximo slide, com `try/except/finally`.
3. Rode **duas vezes**: as duas mostram as três mensagens, sem *ACONTECEU UM ERRO*.
4. Confira no pgAdmin: 5 filmes e 6 sessões.

PRÁTICA 1 • DADOS PARA LANÇAR E TESTES

O setup do Cinema Estrela

FILMES • NA ORDEM: O BANCO GERA 1 A 5

titulo_filme	duracao	classif.
O Último Farol	112	12
Corrida nas Dunas	98	L
Código Vermelho	125	16
A Ilha dos Gatos	90	L
Noite sem Fim	105	14

SESSOES • NA ORDEM: O BANCO GERA 1 A 6

cod_filme	data_sessao	horario	sala
1	2026-11-06	19:00	1
1	2026-11-06	21:30	1
2	2026-11-07	15:00	2
3	2026-11-07	20:00	3
4	2026-11-08	14:00	2
2	2026-11-08	16:30	4

PROVOQUE DOIS ERROS, UM DE CADA VEZ

1. Troque uma letra do `DB_PASSWORD` no `.env`. Rode. O que o `except` mostrou? Algum `print` do `try` apareceu? Desfaça.
2. Escreva `CREAT` no CREATE de `sessoes`. Rode. Até qual `print` o script chegou? No pgAdmin, os filmes da rodada anterior continuam lá? Desfaça.

Anote as duas mensagens como comentário no fim do `setup_cinema.py`.

PENSE E RESPONDA No erro 2, o DROP de sessoes e o DROP de filmes rodaram antes do CREAT. Por que os filmes continuam no banco?

SEÇÃO 02

O primeiro sistema

02

Um menu no terminal que consulta e cadastra: a conexão em cada opção e os valores digitados por %s.

O Sistema RH, ainda sem classe

1. Criar um arquivo `sistema_rh.py`, na mesma pasta do `setup_rh.py` e do `.env`.
2. Criar o menu no terminal, com as opções **1 • Ver Funcionários**, **2 • Ver Departamentos** e **0 • Sair**.
3. Criar a infraestrutura de conexão com o banco em cada opção: o mesmo molde do setup.
4. Expandir com **3 • Cadastrar Novo Funcionário** e **4 • Cadastrar Novo Departamento**.

> Cada opção abre a **própria** conexão, roda o SQL e fecha. Funciona, mas repete muito código: guarde essa sensação para a Seção 03.

TERMINAL

```
> python sistema_rh.py

    SEJA BEM VINDO AO SISTEMA RH

    MENU:

    1 - Ver Funcionários
    2 - Ver Departamentos
    3 - Cadastrar Novo Funcionário
    4 - Cadastrar Novo Departamento

    0 - Sair

Digite a opção desejada: 1
```

Ver funcionários: SELECT e fetchall

```
if (op == "1"):
    print("VER FUNCIONÁRIOS")

    funcionarios = None
    conexaoDB = None
    cursor = None

    try:
        conexaoDB = psycopg.connect(...)   # os 6 dados do .env, como no setup
        cursor = conexaoDB.cursor()

        cursor.execute("SELECT * FROM funcionarios;")

        funcionarios = cursor.fetchall()

    except Exception as error:
        print("OCORREU UM ERRO:", error)
        if (conexaoDB is not None):
            conexaoDB.rollback()
    finally:
        if (cursor is not None):
            cursor.close()
        if (conexaoDB is not None):
            conexaoDB.close()

    print("LISTA DE FUNCIONÁRIOS")
    print("ID | NOME | CPF | CARGO | SALÁRIO | COD DEPT")

    if (funcionarios):
        for func in funcionarios:
            print(f"{func[0]} | {func[1]} | {func[2]} | {func[3]} | R$ {func[4]:,.2f} | {func[5]}")
    else:
        print("ERRO NA CONSULTA")
```

TERMINAL

```
VER FUNCIONÁRIOS
LISTA DE FUNCIONÁRIOS
ID | NOME | CPF | CARGO | SALÁRIO | COD DEPT
1 | Ana Souza | 12345678901 | Analista de RH | R$ 4,200.00
| 1
2 | Bruno Lima | 23456789012 | Vendedor | R$ 3,100.00 | 2
3 | Carla Mendes | 34567890123 | Desenvolvedora | R$
6,800.00 | 3
4 | Diego Rocha | 45678901234 | Suporte técnico | R$
3,500.00 | 3
5 | Elisa Martins | 56789012345 | Gerente comercial | R$
7,500.00 | 2
```

- O molde do setup, agora dentro de uma opção do menu.
- `fetchall()` traz todas as linhas da resposta, como a grade do pgAdmin.
- Deu erro? `funcionarios` continua `None`, e o programa avisa: *ERRO NA CONSULTA*.

Linhas, tuplas e tipos

```
funcionarios = cursor.fetchall()
print(funcionarios[0])
print(funcionarios[0][1])
```

TERMINAL

```
(1, 'Ana Souza', '12345678901', 'Analista de RH',
Decimal('4200.00'), 1)
Ana Souza
```

POSIÇÃO	COLUNA	POSIÇÃO	COLUNA
`func[0]`	cod_funcionario	`func[3]`	cargo_funcionario
`func[1]`	nome_funcionario	`func[4]`	salario_funcionario
`func[2]`	cpf_funcionario	`func[5]`	cod_setor

Cada linha é uma **tupla**. Com `SELECT *`, as posições seguem a ordem do `CREATE TABLE`. Na UC 1 era `funcionario["nome"]`; aqui, `func[1]`.

NO POSTGRESQL	NO PYTHON	EXEMPLO
`integer`	`int`	`1`
`numeric(10,2)`	`Decimal`	`Decimal('4200.00')`
`varchar` • `char`	`str`	`'Ana Souza'`
`date`	`datetime.date`	`date(2026, 11, 6)`
`time`	`datetime.time`	`time(19, 0)`
`NULL`	`None`	`None`

- `Decimal` guarda dinheiro sem erro de arredondamento. A f-string formata igual: `f"R$ {func[4]:,.2f}"`.
- Data e hora no formato brasileiro: `f"{data:%d/%m/%Y}"` e `f"{horario:%H:%M}"`.

O valor vem do usuário

No cadastro, os dados são **digitados**:

```
nome = input("Digite o nome do Funcionário: ")
cpf = input("Digite o cpf do Funcionário: ")
salario = float(input("Digite o salário do Funcionário: "))
```

A primeira ideia é colar os valores no SQL com f-string:

```
cursor.execute(f"INSERT INTO funcionarios VALUES (default, '{nome}', ...)")
```

- Texto precisa de aspas simples, e esquecer dá erro.
- Um nome como **Joana D'Ávila** tem um apóstrofo, que fecha a aspa antes da hora.
- Quem digita pode **reescrever o seu SQL**: SQL injection.

O jeito certo: parâmetros

```
cursor.execute('''
    INSERT INTO funcionarios
    VALUES (default, %s, %s, %s, %s, %s);
''', (nome, cpf, cargo, salario, cod_setor))
```

Cada `%s` marca o lugar de um valor. Os valores vão separados, numa tupla, e o psycopg entrega os dois ao servidor sem misturar.

Parâmetros: o molde

```
cursor.execute('''
    INSERT INTO tabela
    VALUES (default, %s, %s, %s);
''', (valor1, valor2, valor3))

cursor.execute("SELECT * FROM tabela WHERE cod = %s;",
               (cod,))
```

1º `%s` ← 1º valor da tupla 2º `%s` ← 2º valor e assim por diante

- `default` no lugar do código: o banco gera o número (`GENERATED ALWAYS`). Sem listar as colunas, os valores seguem a **ordem do CREATE TABLE**.
- `%s` serve para **qualquer tipo**: número, texto, data, `None`.
- **Sem aspas** em volta do `%s`, mesmo para texto.
- Quantos `%s`, tantos valores na tupla, na mesma ordem.
- Um valor só? A vírgula é obrigatória: `(cod,)`. Sem ela, `(cod)` é só um número entre parênteses.

Cadastrar funcionário: INSERT com %s

```python
elif (op == "3"):
    print("CADASTRO DE FUNCIONÁRIO")

    nome = input("Digite o nome do Funcionário: ")
    cpf = input("Digite o cpf do Funcionário: ")
    cargo = input("Digite o cargo do Funcionário: ")
    salario = float(input("Digite o salário do Funcionário: "))
    cod_setor = int(input("Digite o código do setor: "))

    conexaoDB = None
    cursor = None

    try:
        conexaoDB = psycopg.connect(...)   # os 6 dados do .env
        cursor = conexaoDB.cursor()

        # default: o banco gera o código (GENERATED ALWAYS)
        cursor.execute('''
            INSERT INTO funcionarios
            VALUES (default, %s, %s, %s, %s, %s);
        ''', (nome, cpf, cargo, salario, cod_setor))

        conexaoDB.commit()
        print("FUNCIONÁRIO CADASTRADO!")

    except Exception as error:
        print("ERRO NO CADASTRO DO FUNCIONÁRIO")
        print(error)
        if (conexaoDB is not None):
            conexaoDB.rollback()
    finally:
        # fecha o cursor e a conexão, como na opção 1
```

TERMINAL

```
CADASTRO DE FUNCIONÁRIO
Digite o nome do Funcionário: Fábio Nunes
Digite o cpf do Funcionário: 67890123456
Digite o cargo do Funcionário: Estagiário
Digite o salário do Funcionário: 1500
Digite o código do setor: 3
FUNCIONÁRIO CADASTRADO!
```

TERMINAL • CPF DA ANA

```
Digite o cpf do Funcionário: 12345678901
...
ERRO NO CADASTRO DO FUNCIONÁRIO
duplicate key value violates unique constraint
"funcionarios_cpf_funcionario_key"
DETAIL:  Key (cpf_funcionario)=(12345678901) already exists.
```

INSERT muda o banco: precisa do `commit()`. O CPF repetido foi recusado pelo `UNIQUE` do Capítulo 4, e o `except` mostrou a mensagem do banco.

Cadastrar departamento: um valor só

```python
elif (op == "4"):
    print("CADASTRAR NOVO DEPARTAMENTO")

    nome = input("DIGITE O NOME DO DEPARTAMENTO: ")

    # Nome vazio vira None (NULL): o NOT NULL do banco recusa
    if (not nome):
        nome = None

    # ... conexaoDB, cursor e o try, como na opção 3

        # Um valor só: a vírgula faz de (nome,) uma tupla
        cursor.execute('''
            INSERT INTO setores
            VALUES (default, %s);
        ''', (nome,))

        conexaoDB.commit()
        print("DEPARTAMENTO CADASTRADO!")

    except Exception as error:
        print("ERRO AO CRIAR DEPARTAMENTO")
        print(error)
```

TERMINAL

```
CADASTRAR NOVO DEPARTAMENTO
DIGITE O NOME DO DEPARTAMENTO: Logística
DEPARTAMENTO CADASTRADO!
```

TERMINAL • NOME VAZIO

```
CADASTRAR NOVO DEPARTAMENTO
DIGITE O NOME DO DEPARTAMENTO:
ERRO AO CRIAR DEPARTAMENTO
null value in column "nome_setor" of relation
"setores" violates not-null constraint
DETAIL:  Failing row contains (5, null).
```

- `(nome,)`: com um valor só, a vírgula é o que faz a tupla.
- Texto vazio `''` não é `NULL` para o banco. Trocando por `None`, o `NOT NULL` recusa.
- O 5 do DETAIL: o `GENERATED` gastou um número mesmo sem gravar.

SQL injection: o usuário reescreve o seu SQL

COM F-STRING: O VALOR COLADO NO TEXTO

```
nome = input("Nome: ")
cursor.execute(f"SELECT nome_funcionario, salario_funcionario
  FROM funcionarios WHERE nome_funcionario = '{nome}'")
```

DIGITADO • O WHERE QUE CHEGA AO BANCO	RESPOSTA
Ana Souza `WHERE nome_funcionario = 'Ana Souza'`	1 linha
' OR '1'='1 `WHERE nome_funcionario = '' OR '1'='1'`	**as 5 linhas, com todos os salários**
Joana D'Ávila `WHERE nome_funcionario = 'Joana D'Ávila'`	`syntax error at or near "Ávila"`

COM %S: O VALOR VAI SEPARADO

```
nome = input("Nome: ")
cursor.execute("""SELECT nome_funcionario, salario_funcionario
  FROM funcionarios WHERE nome_funcionario = %s""", (nome,))
```

DIGITADO	RESPOSTA
`Ana Souza`	1 linha
`' OR '1'='1`	`[]`: procurou alguém com esse nome esquisito
`Joana D'Ávila`	`[]`, sem erro

⚠ No DELETE, o mesmo truque apaga a tabela inteira. No teste, `f"DELETE FROM funcionarios WHERE cpf_funcionario = '{cpf}'"` com `' OR '1'='1` apagou os 5 funcionários. Regra do capítulo: **valor digitado entra só por %s**.

EXEMPLO EXPLICADO • ERROS COMUNS COM PARÂMETROS

Cinco jeitos de errar o %s

TRECHO	O QUE O EXCEPT MOSTRA	CORREÇÃO
`... WHERE cod_setor = %s", (cod))`	`object of type 'int' has no len()`	Sem vírgula, `(cod)` não é tupla: `(cod,)`
`... WHERE nome_setor = '%s'", (nome,))`	`could not determine data type of parameter $1`	Sem aspas em volta do `%s`
`... WHERE cod_setor = %d", (cod,))`	`only '%s', '%b', '%t' are allowed as placeholders, got '%d'`	Só `%s`, para qualquer tipo
`SET ... = %s WHERE cod = %s", (valor,))`	`the query has 2 placeholders but 1 parameters were passed`	Um valor para cada `%s`: `(valor, cod)`
`f"... WHERE titulo_filme = {titulo}"`	`syntax error at or near "sem"`	O texto “Noite sem Fim” entrou sem aspas no SQL: use `%s`

Ler a mensagem do `except` resolve quase todos: ela diz o que o psycopg ou o banco não entendeu.

EXEMPLO EXPLICADO • MENOS REPETIÇÃO

A conexão repetida vira duas funções

sistema_cinema.py

```python
def consultarBanco(sql, parametros=()):
    conexaoDB = None
    cursor = None

    try:
        conexaoDB = psycopg.connect(
            host=os.getenv("DB_HOST"),
            port=os.getenv("DB_PORT"),
            dbname="cinema",
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            sslmode=os.getenv("DB_SSL")
        )
        cursor = conexaoDB.cursor()

        cursor.execute(sql, parametros)

        return cursor.fetchall()
    except Exception as error:
        print("ERRO AO CONSULTAR BANCO")
        print(error)
        if (conexaoDB is not None):
            conexaoDB.rollback()
        return None
    finally:
        if (cursor is not None):
            cursor.close()
        if (conexaoDB is not None):
            conexaoDB.close()
```

No `sistema_rh.py`, as quatro opções repetem o mesmo bloco de conexão. No sistema do cinema, ele vai para duas funções:

FUNÇÃO	PARA	DEVOLVE
`consultarBanco(sql, parametros)`	SELECT	as linhas, ou `None` se deu erro
`manipularBanco(sql, parametros)`	INSERT, UPDATE, DELETE	`True` com `commit()`, ou `False`

```python
filmes = consultarBanco("SELECT * FROM filmes;")

manipularBanco('''
    INSERT INTO filmes VALUES (default, %s, %s, %s);
''', (titulo, duracao, classificacao))
```

Cada opção do menu cai para poucas linhas. Na Seção 03, as duas funções viram os métodos de uma classe.

PRÁTICA 2 • 30 MIN • INDIVIDUAL

O sistema do Cinema Estrela

O mesmo que o `sistema_rh.py`, no banco **cinema** do `setup_cinema.py`: crie o `sistema_cinema.py`.

1. Menu com **1 • Ver Filmes**, **2 • Ver Sessões**, **3 • Cadastrar Novo Filme**, **4 • Cadastrar Nova Sessão** e **0 • Sair**.
2. Conexão com `dbname="cinema"`, o `.env` do RH e `try/except/finally`. Use as funções `consultarBanco` e `manipularBanco` para não repetir.
3. Cadastros com `VALUES (default, %s, …)` e os valores numa tupla.

FUNCIONALIDADES EXTRAS

- **Ver detalhes de um filme**: mostre a lista, peça o código e exiba só aquele filme.
- Em **Ver Sessões**, mostre o **título do filme** no lugar do código: *JOIN*.

filmes

PK	**cod_filme**
	titulo_filme
	duracao_filme
	classificacao_filme

sessoes

PK	**cod_sessao**
FK	cod_filme
	data_sessao
	horario_sessao
	sala_sessao

A ordem das colunas é a do `CREATE TABLE` do `setup_cinema.py`: é a ordem dos `%s` depois do `default`.

PRÁTICA 2 • TESTES

O sistema do Cinema Estrela

TESTE	ESPERADO
1 • Ver Filmes, logo depois do setup	Os 5 filmes, de O Último Farol a Noite sem Fim
2 • Ver Sessões	As 6 sessões, cada uma com o título do filme: *1 | O Último Farol | 06/11/2026 | 19:00 | 1*
3 • Maré Alta • 101 • 12	Filme cadastrado; ele aparece como 6 em Ver Filmes
3 • Sem Duração • 0 • L	O `except` mostra *violates check constraint "ck_duracao"*; nada entra
4 • filme 5 • 2026-11-09 • 18:00 • sala 3	Sessão cadastrada
4 • filme 5 • 2026-11-09 • 18:00 • sala 9	O `except` mostra *violates check constraint "ck_sala"*
Detalhes • filme 2	Código 2, Corrida nas Dunas, 98 minutos, classificação L
Detalhes • filme 99	Uma mensagem de filme não encontrado, sem Traceback

Gabarito no repositório: `cinema/sistema_cinema.py`.

PENSE E RESPONDA No filme 99, a consulta não deu erro nenhum, mas a lista voltou vazia. Como o programa percebe isso antes de tentar mostrar o filme?

SEÇÃO 03

A classe Banco e o CRUD

03

A conexão num lugar só e o CRUD completo do RH: ver, ver detalhes, cadastrar, alterar e remover.

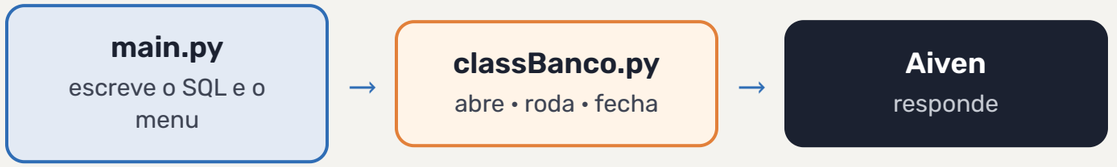
03 • CLASSE • A IDEIA

Das funções para a classe

As funções `consultarBanco` e `manipularBanco` já tiraram a repetição. A classe **Banco** junta as duas, com os dados da conexão, num arquivo só, que qualquer sistema pode importar.

NA PRÁTICA 2 • FUNÇÃO	NA CLASSE BANCO • MÉTODO	DEVOLVE
`consultarBanco(sql, parametros)`	`banco.consultar(sql, parametros)`	a lista de linhas
`manipularBanco(sql, parametros)`	`banco.manipular(sql, parametros)`	quantas linhas mudaram
o `connect` dentro de cada função	`banco.conectar()`	uma conexão nova

Como a classe `Conta` da POO: os **atributos** guardam os dados da conexão, e os **métodos** fazem o trabalho.




```
Sistema RH - CRUD/
├── .env
├── setup_rh.py
├── classBanco.py   a classe Banco
└── main.py         o CRUD, que usa a classe
```

classBanco.py: os dados da conexão

classBanco.py

```python
import os
import psycopg
from dotenv import load_dotenv

load_dotenv()


class Banco():
    def __init__(self):
        self.host = os.getenv("DB_HOST")
        self.port = os.getenv("DB_PORT")
        self.user = os.getenv("DB_USER")
        self.password = os.getenv("DB_PASSWORD")
        self.dbname = os.getenv("DB_NAME")
        self.ssl = os.getenv("DB_SSL")

    # Esse método deveria ser privado, ou seja, executado somente dentro da classe
    def conectar(self):
        return psycopg.connect(
            host=self.host,
            port=self.port,
            user=self.user,
            password=self.password,
            dbname=self.dbname,
            sslmode=self.ssl
        )
```

- O arquivo segue o padrão da POO: `classBanco.py`, e no programa `from classBanco import Banco`.
- O `__init__` copia o `.env` para os atributos: `Banco()` não precisa de nenhum argumento.
- `conectar()` abre e devolve uma conexão nova. Só os métodos da própria classe usam: por isso o comentário de que ele deveria ser privado.

classBanco.py: consultar

```python
    # SELECT: devolve a lista de linhas (cada linha é uma tupla)
    def consultar(self, sql, parametros=()):
        conexao = None

        try:
            conexao = self.conectar()
            cursor = conexao.cursor()

            cursor.execute(sql, parametros)

            return cursor.fetchall()

        except Exception as error:
            print("ERRO DE CONSULTA:", error)
            raise

        finally:
            if (conexao is not None):
                conexao.close()
```

- O molde do setup, dentro de um método: `conexao = None`, `try`, `except` e `finally`.
- `parametros=()`: o argumento é opcional. Um SELECT sem valores digitados não precisa dele.
- O `return` não pula o `finally`: o Python fecha a conexão e só depois devolve as linhas.
- O `except` mostra o erro e o `raise` repassa (dois slides adiante).

classBanco.py: manipular

```python
    # INSERT, UPDATE ou DELETE: grava e devolve quantas linhas mudaram
    def manipular(self, sql, parametros=()):
        conexao = None

        try:
            conexao = self.conectar()
            cursor = conexao.cursor()

            resultado = cursor.execute(sql, parametros)

            conexao.commit()

            return resultado.rowcount

        except Exception as error:
            print("ERRO DE MANIPULAÇÃO:", error)
            raise

        finally:
            if (conexao is not None):
                conexao.close()
```

- Igual ao `consultar`, com duas diferenças: o `commit()` e o que devolve.
- O `execute` devolve o próprio cursor; o `rowcount` dele diz quantas linhas o INSERT, o UPDATE ou o DELETE mudou.
- Deu erro antes do `commit()`? Fechar a conexão sem gravar desfaz o que ficou pendente.

Arquivo completo no repositório: `rh/classBanco.py`.

EXEMPLO EXPLICADO • O EXCEPT DA CLASSE

A classe mostra o erro e repassa

```
        except Exception as error:
            print("ERRO DE MANIPULAÇÃO:", error)
            raise
```

- O `print` mostra a mensagem do banco no terminal: serve para quem está programando achar o problema (**debug**).
- `raise` sozinho **repassa o mesmo erro** para quem chamou. Sem ele, a classe engoliria o erro e o cadastro diria que deu certo.
- Quem chamou trata com o próprio `try/except` e mostra uma mensagem para o usuário.

```
try:
    banco.manipular('''
        INSERT INTO funcionarios
        VALUES (default, %s, %s, %s, %s, %s);
    ''', (nome, cpf, cargo, salario, cod_setor))
    print("FUNCIONÁRIO CADASTRADO COM SUCESSO!")
except Exception as error:
    print("ERRO AO CADASTRAR FUNCIONÁRIO")
```

TERMINAL • CPF DA ANA

```
Digite o cpf do funcionário: 12345678901
...
ERRO DE MANIPULAÇÃO: duplicate key value violates unique constraint
"funcionarios_cpf_funcionario_key"
DETAIL:  Key (cpf_funcionario)=(12345678901) already exists.
ERRO AO CADASTRAR FUNCIONÁRIO
```

Em cinza, a mensagem da classe; em branco, a do programa.

Usando a classe

```python
from classBanco import Banco

banco = Banco()      # lê o .env uma vez

setores = banco.consultar("SELECT * FROM setores ORDER BY cod_setor;")
print(setores)

for cod_setor, nome_setor in setores:
    print(f"{cod_setor} | {nome_setor}")

linhas = banco.manipular(
    "DELETE FROM funcionarios WHERE cod_funcionario = %s;", (99,))
print(linhas, "linha(s) apagada(s)")
```

TERMINAL

```
> python teste_classe.py
[(1, 'Administrativo'), (2, 'Comercial'), (3, 'Tecnologia')]
1 | Administrativo
2 | Comercial
3 | Tecnologia
0 linha(s) apagada(s)
```

- Uma linha por consulta: conectar, cursor e fechar ficam escondidos na classe.
- `for cod_setor, nome_setor in setores` desempacota cada tupla: uma variável por coluna, na ordem do SELECT. Também vale `setor[0]`, `setor[1]`.
- O DELETE do funcionário 99 não deu erro: ninguém tinha esse código, e o `rowcount` foi **0**.

03 • PARA SABER

Duas formas que você vai encontrar por aí

LINHAS COMO DICIONÁRIO: DICT_ROW

```python
from psycopg.rows import dict_row

conexao = banco.conectar()
cursor = conexao.cursor(row_factory=dict_row)
cursor.execute("""SELECT nome_funcionario, salario_funcionario
                  FROM funcionarios WHERE cod_funcionario = 3""")
print(cursor.fetchall())
conexao.close()
```

```
TERMINAL
[{'nome_funcionario': 'Carla Mendes', 'salario_funcionario':
Decimal('6800.00')}]
```

Igual à lista de dicionários da UC 1. Neste capítulo seguimos com tuplas: o `for` com desempacotamento resolve com menos código.

A CONEXÃO COM WITH

```python
with banco.conectar() as conexao:
    conexao.execute(
        "DELETE FROM funcionarios WHERE cod_funcionario = %s", (6,))
# saiu do with: commit feito e conexão fechada
```

- No fim do bloco, o `with` faz o `commit` e fecha. Se deu erro, faz o `rollback` e fecha.
- É o mesmo papel do `try/finally`, escrito de outro jeito, como o `with open(...)` do `dados.json`.
- Neste capítulo usamos o `try/except/finally`: cada passo fica à vista.

Da lista para a tabela

NA UC 1 • LISTA E DADOS.JSON	COM A CLASSE BANCO • TABELA FUNCIONARIOS
`funcionarios = carregar_dados()`	`banco = Banco()` : os dados ficam no servidor
`funcionarios.append(novo_funcionario)`	`banco.manipular("INSERT INTO funcionarios VALUES (default, %s, ...)", ...)`
`for funcionario in funcionarios: print(...)`	`banco.consultar("SELECT ... JOIN setores ...")` e o `for` nas linhas
`funcionario["cargo"] = novo_cargo`	`banco.manipular("UPDATE funcionarios SET ... WHERE cod_funcionario = %s", ...)`
`funcionarios.pop(num_funcionario - 1)`	`banco.manipular("DELETE FROM funcionarios WHERE cod_funcionario = %s", ...)`
escolher pela **posição** na lista	escolher pelo **código**, a PK
`cpf_exist(cpf)` percorre a lista	o `UNIQUE` recusa, e o `except` avisa
`salvar_dados()` só ao sair	cada `manipular` já grava: o `commit` está dentro dele

O menu e o `while True` continuam iguais. Muda só **onde o dado mora**.

main.py: o menu do CRUD

```
while (True):
    print('''
    BEM VINDO AO SISTEMA RH XYZ
    MENU:
    1. Ver Funcionários
    2. Escolher Funcionário
    3. Cadastrar Funcionário
    4. Alterar Funcionário
    5. Remover Funcionário
    11. Ver Setores
    ...
    0. Sair
    ''')

    op = input("DIGITE O NÚMERO DA OPÇÃO DESEJADA: ")

    if (op == "1"):
        verFuncionarios()
    elif (op == "2"):
        escolherFuncionario()
    # ... 3, 4, 5 e 11 a 14
    elif (op == "0"):
        print("Saindo do programa...")
        break
    else:
        print("ESCOLHA UMA OPÇÃO VÁLIDA")

    input("TECLE ENTER PARA CONTINUAR")
```

OPÇÃO	FUNÇÃO	O QUE FAZ
1	`verFuncionarios()`	lista todos, com o nome do setor
2	`escolherFuncionario()`	pede o código e mostra a ficha: **ver detalhes**
3	`cadastrarFuncionario()`	INSERT
4	`alterarFuncionario()`	UPDATE, deixando vazio o que não muda
5	`removerFuncionario()`	DELETE, depois de confirmar
11 a 14	`verSetores()` …	o mesmo para os setores

No começo do `main.py`: `from classBanco import Banco` e `banco = Banco()`, um objeto só para todas as funções.

verFuncionarios: SELECT com JOIN

```python
def verFuncionarios():
    print("LISTA DE FUNCIONÁRIOS")
    funcionarios = banco.consultar('''
        SELECT f.cod_funcionario, f.nome_funcionario, f.cpf_funcionario, f.cargo_funcionario,
               f.salario_funcionario, f.cod_setor, s.nome_setor
        FROM funcionarios f
        JOIN setores s ON f.cod_setor = s.cod_setor
        ORDER BY f.cod_funcionario ASC;
    ''')
    print("Código | Nome | CPF | Cargo | Salário | Cod Setor | Setor")
    if (len(funcionarios) == 0):
        print("NÃO HÁ FUNCIONÁRIOS CADASTRADOS")
        return
    # Alternativa: for func in funcionarios, e func[0], func[1]...
    for cod_func, nome_func, cpf_func, cargo_func, salario_func, cod_setor, nome_setor in funcionarios:
        print(f"{cod_func} | {nome_func} | {cpf_func} | {cargo_func} | R$ {salario_func:,.2f} | {cod_setor} | {nome_setor}")
```

TERMINAL

```
LISTA DE FUNCIONÁRIOS
Código | Nome | CPF | Cargo | Salário | Cod Setor | Setor
1 | Ana Souza | 12345678901 | Analista de RH | R$ 4,200.00 | 1 |
Administrativo
2 | Bruno Lima | 23456789012 | Vendedor | R$ 3,100.00 | 2 | Comercial
...
```

- O JOIN do Capítulo 4 traz o **nome do setor**.
- `len(funcionarios) == 0`: o banco respondeu, mas sem linhas. Não é erro.
- O `for` desempacota as 7 colunas, na ordem do SELECT.

escolherFuncionario: a ficha de um só

```python
def escolherFuncionario():
    verFuncionarios()

    cod_escolhido = int(input("Digite o código do funcionário desejado: "))

    consulta = banco.consultar('''
        SELECT f.cod_funcionario, f.nome_funcionario, f.cpf_funcionario, f.cargo_funcionario,
               f.salario_funcionario, f.cod_setor, s.nome_setor
        FROM funcionarios f
        JOIN setores s ON f.cod_setor = s.cod_setor
        WHERE f.cod_funcionario = %s;
    ''', (cod_escolhido,))

    if (len(consulta) == 0):
        print("Funcionário não encontrado!")
        return None

    funcionario = consulta[0]

    print(f'''
    Ficha do Funcionário:

    Código: {funcionario[0]}
    Nome: {funcionario[1]}
    CPF: {funcionario[2]}
    Cargo: {funcionario[3]}
    Salário: R$ {funcionario[4]:,.2f}
    Setor: {funcionario[6]}
    ''')

    return funcionario
```

TERMINAL

```
Digite o código do funcionário desejado: 3

    Ficha do Funcionário:

    Código: 3
    Nome: Carla Mendes
    CPF: 34567890123
    Cargo: Desenvolvedora
    Salário: R$ 6,800.00
    Setor: Tecnologia
```

- O código digitado entra por `%s`, com a vírgula: `(cod_escolhido,)`.
- Código 99? A lista volta vazia, e a função avisa e devolve `None`.
- O `return funcionario` deixa a mesma função servir ao alterar e ao remover.

cadastrarFuncionario: INSERT pela classe

```python
def cadastrarFuncionario():
    print("CADASTRO DE NOVO FUNCIONÁRIO")

    nome = input("Digite o nome do funcionário: ")
    cpf = input("Digite o cpf do funcionário: ")
    cargo = input("Digite o cargo do funcionário: ")
    salario = float(input("Digite o salário do funcionário: "))

    verSetores()

    cod_setor = int(input("Digite o código do setor do funcionário: "))

    try:
        banco.manipular('''
            INSERT INTO funcionarios
            VALUES (default, %s, %s, %s, %s, %s);
        ''', (nome, cpf, cargo, salario, cod_setor))
        print("FUNCIONÁRIO CADASTRADO COM SUCESSO!")
    except Exception as error:
        print("ERRO AO CADASTRAR FUNCIONÁRIO")
```

TERMINAL

```
CADASTRO DE NOVO FUNCIONÁRIO
Digite o nome do funcionário: Fábio Nunes
Digite o cpf do funcionário: 67890123456
Digite o cargo do funcionário: Estagiário
Digite o salário do funcionário: 1500
LISTA DE SETORES
...
Digite o código do setor do funcionário: 3
FUNCIONÁRIO CADASTRADO COM SUCESSO!
```

- O mesmo INSERT da Seção 02, agora numa linha: `banco.manipular(...)`.
- `verSetores()` mostra os códigos antes da pergunta.
- O `try` fica em quem chama: CPF repetido mostra *ERRO AO CADASTRAR FUNCIONÁRIO*.

alterarFuncionario: deixe vazio para manter

```python
def alterarFuncionario():
    print("ATUALIZAÇÃO DE FUNCIONÁRIO")
    funcionario = escolherFuncionario()
    if (not funcionario):
        print("Não é possível alterar o funcionário. Cheque o código escolhido...")
        return
    continuar = input("Deseja alterar as informações do funcionário? (S/N) ")
    if (continuar != "S"):
        return
    cod, nome, cpf, cargo, salario, cod_setor, nome_setor = funcionario
    print("ALTERAR DADOS:")
    print("(Deixe vazio para manter o original)")
    novo_nome = input(f"Digite o novo nome ({nome}): ").strip()
    if (novo_nome):
        nome = novo_nome
    novo_cpf = input(f"Digite o novo cpf ({cpf}): ").strip()
    if (novo_cpf):
        cpf = novo_cpf
    novo_cargo = input(f"Digite o novo cargo ({cargo}): ").strip()
    if (novo_cargo):
        cargo = novo_cargo
    novo_salario = input(f"Digite o novo salário (R$ {salario:,.2f}): ").strip()
    if (novo_salario):
        salario = float(novo_salario)
    verSetores()
    novo_setor = input(f"Digite o código do novo setor ({cod_setor}): ").strip()
    if (novo_setor):
        cod_setor = int(novo_setor)
```

TERMINAL

```
Deseja alterar as informações do funcionário? (S/N) S
ALTERAR DADOS:
(Deixe vazio para manter o original)
Digite o novo nome (Fábio Nunes):
Digite o novo cpf (67890123456):
Digite o novo cargo (Estagiário): Desenvolvedor júnior
Digite o novo salário (R$ 1,500.00): 2800
...
Digite o código do novo setor (3):
```

- `escolherFuncionario()` mostra a ficha e devolve a linha, ou `None`.
- A confirmação **S/N** evita alterar o funcionário errado.
- A linha é desempacotada com os valores **atuais**; o que vier digitado substitui, o vazio mantém.

alterarFuncionario: o UPDATE

```
try:
    banco.manipular('''
        UPDATE funcionarios
        SET nome_funcionario = %s,
            cpf_funcionario = %s,
            cargo_funcionario = %s,
            salario_funcionario = %s,
            cod_setor = %s
        WHERE cod_funcionario = %s;
    ''', (nome, cpf, cargo, salario, cod_setor, cod))
    print("FUNCIONÁRIO ATUALIZADO!")
except Exception as error:
    print("Ocorreu um erro na atualização!")
```

TERMINAL

```
...
Digite o código do novo setor (3):
FUNCIONÁRIO ATUALIZADO!
```

- O UPDATE regrava todas as colunas, com os valores novos ou os mantidos.
- Seis `%s`, seis valores: o código vai no último, o do WHERE.
- CPF com mais de 11 dígitos? A classe mostra *value too long for type character(11)*, e o programa, *Ocorreu um erro na atualização!*

removerFuncionario: confirmar e apagar

```python
def removerFuncionario():
    print("REMOÇÃO DE FUNCIONÁRIO")

    funcionario = escolherFuncionario()

    if (not funcionario):
        print("Funcionário não encontrado!")
        return

    continuar = input("DESEJA REALMENTE APAGAR ESSE FUNCIONÁRIO? (S/N) ")

    if (continuar != "S"):
        print("Cancelando operação...")
        return

    try:
        banco.manipular('''
            DELETE FROM funcionarios
            WHERE cod_funcionario = %s;
        ''', (funcionario[0],))
        print("Funcionário removido com sucesso!")
    except Exception as error:
        print("Não foi possível remover o funcionário!")
```

```
TERMINAL
REMOÇÃO DE FUNCIONÁRIO
(a lista e a ficha do funcionário 6)
DESEJA REALMENTE APAGAR ESSE FUNCIONÁRIO? (S/N) S
Funcionário removido com sucesso!
```

- A confirmação **S/N** é do Python: o DELETE não tem volta.
- `funcionario[0]` é o código, a primeira coluna do SELECT.
- Código que não existe? `escolherFuncionario` devolve `None`, e a remoção para ali.

O mesmo caminho para os setores

```python
def escolherSetor():
    verSetores()
    id_setor = int(input("DIGITE O CÓDIGO DO SETOR DESEJADO: "))
    consulta = banco.consultar('''
        SELECT cod_setor, nome_setor FROM setores
        WHERE cod_setor = %s;
    ''', (id_setor,))
    if (len(consulta) == 0):
        print("SETOR NÃO ENCONTRADO")
        return None
    setor = consulta[0]
    funcionarios = banco.consultar('''
        SELECT cod_funcionario, nome_funcionario, cargo_funcionario FROM funcionarios
        WHERE cod_setor = %s;
    ''', (id_setor,))
    print(f"FUNCIONÁRIOS DO SETOR {setor[0]} - {setor[1]}:")
    if (len(funcionarios) == 0):
        print("(nenhum funcionário)")
    for func in funcionarios:
        print(f"{func[0]} - {func[1]} - {func[2]}")
    return setor
```

OPÇÃO	FUNÇÃO
11	`verSetores()` : `SELECT * FROM setores`
12	`escolherSetor()` : o setor e os funcionários dele
13	`cadastrarSetor()` : confere o nome vazio antes do INSERT
14	`alterarSetor()` : escolhe, pede o novo nome e faz o UPDATE

TERMINAL

```
DIGITE O CÓDIGO DO SETOR DESEJADO: 3
FUNCIONÁRIOS DO SETOR 3 - Tecnologia:
3 - Carla Mendes - Desenvolvedora
4 - Diego Rocha - Suporte técnico
```

Duas consultas: uma confere se o setor existe; a outra traz os funcionários. Um setor novo, sem ninguém, aparece com *(nenhum funcionário)*.

Quem valida: o Python ou o banco?

O PYTHON • A PRIMEIRA PORTA

- Campo vazio: `''` passa no NOT NULL. O `cadastrarSetor` confere antes.
- Confirmação **S/N** antes de apagar.
- Código que não existe: a lista volta vazia, e a função avisa.

O BANCO • A ÚLTIMA PORTA

- CPF repetido: só o banco conhece **todos** os CPFs.
- Setor que não existe: a FK.
- CPF com 11 dígitos, salário maior que zero: os CHECKs.
- Vale para **qualquer** programa: o seu, o pgAdmin, o do colega.

Na UC 1, o `cpf_exist` percorria a lista. Com banco, a regra é **uma só**, no servidor: mesmo que o Python esqueça uma validação, a regra segura, e o `except` avisa.

PRÁTICA 3 • 40 MIN • INDIVIDUAL

O CRUD do Cinema Estrela

O mesmo que o `main.py` do RH, para a tabela **filmes**. Numa pasta nova, com o `classBanco.py` e um `.env` com `DB_NAME=cinema`.

1. Implementar o `classBanco.py`: copie o do RH.
2. Implementar o menu principal do CRUD e o objeto `banco = Banco()`.
3. Criar a função `verFilmes` e testar pelo menu.
4. Criar a função `escolherFilme`: pede o código a partir da lista e mostra os detalhes do filme. **Desafio:** mostre também as sessões daquele filme.
5. Completar o CRUD: `cadastrarFilme`, `alterarFilme` (deixe vazio para manter) e `removerFilme` (com S/N).

O MENU

```
CINEMA ESTRELA

1. Ver Filmes
2. Escolher Filme
3. Cadastrar Filme
4. Alterar Filme
5. Remover Filme

11. Ver Sessões

0. Sair
```

O setup é o mesmo da Prática 1: rode o `setup_cinema.py` antes de testar.

PRÁTICA 3 • TESTES

O CRUD do Cinema Estrela

TESTE	ESPERADO
2 • filme 1	A ficha de O Último Farol e as sessões: 06/11/2026 às 19:00 e às 21:30, sala 1
2 • filme 5	A ficha de Noite sem Fim, com *(nenhuma sessão)*
3 • Maré Alta • 101 • 12	*FILME CADASTRADO COM SUCESSO!*, e ele aparece como 6
3 • Maré Alta • 101 • 13	A classe mostra *violates check constraint "ck_classificacao"*, e o programa, *ERRO AO CADASTRAR FILME*
4 • filme 6 • S • só a duração: 95	*FILME ATUALIZADO!*; o título e a classificação continuam os mesmos
5 • filme 1 • S	A classe mostra *violates foreign key constraint "fk_sessao_filme"*; o filme continua
5 • filme 6 • S	*Filme removido com sucesso!*

Gabarito no repositório: `cinema/classBanco.py` e `cinema/main.py`.

PENSE E RESPONDA O filme 6 saiu e o filme 1 não. Que regra do banco decidiu isso, e em qual tabela ela está?

SEÇÃO 04

Prática guiada: Loja Conecta

04

Dois requisitos: o CRUD completo de produtos e o fluxo de venda, com o pedido e os itens gravados juntos.

PRÁTICA GUIADA • OS REQUISITOS

O que a Loja Conecta precisa

O banco do JOIN do Capítulo 4, com os mesmos dados, ganha um sistema em Python. Dois requisitos, nesta ordem:

REQUISITO 1 • CRUD COMPLETO DE PRODUTOS

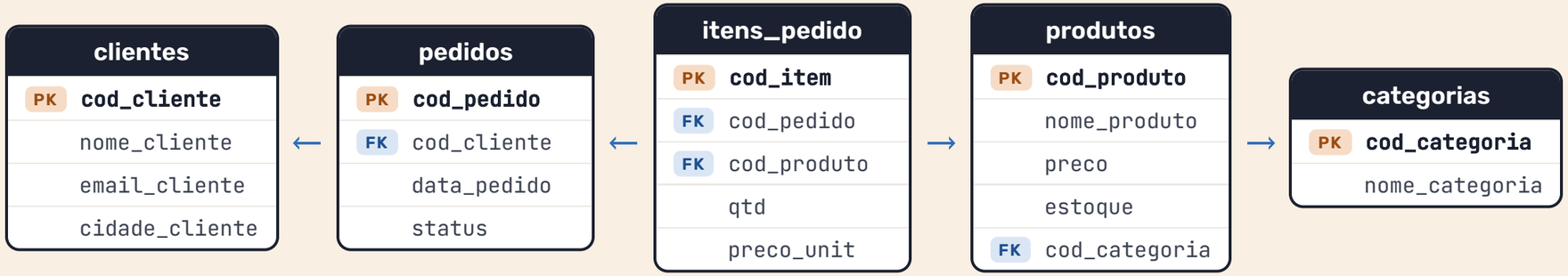
1. **Ver produtos**, com o nome da categoria
2. **Ver detalhes**: a ficha de um produto escolhido
3. **Cadastrar** um produto novo
4. **Alterar**, deixando vazio o que não muda
5. **Apagar**, depois de confirmar

O mesmo caminho do CRUD do RH, na tabela `produtos`.

REQUISITO 2 • FLUXO DE VENDA

1. Escolher o **cliente**
2. Montar a lista de **itens**: produto, quantidade e o preço do dia
3. Gravar o **pedido** e, na sequência, os **itens**, com um método novo na classe: `registrarVenda`
4. **Ver a venda**, com os itens e o total

O pedido e os itens entram juntos, ou nada entra.



PRÁTICA GUIADA • ANTES DE COMEÇAR • 15 MIN

O banco, a classe e o menu

1. Pasta **loja** com um `.env` com `DB_NAME=loja`.
2. Rode o `setup_loja.py` do repositório, duas vezes. É a loja do Capítulo 4, com uma mudança: o banco gera os códigos.
3. Copie o `classBanco.py` do RH.
4. Comece o `main.py`: `from classBanco import Banco`, `banco = Banco()` e o menu ao lado.

```
cod_produto INTEGER GENERATED ALWAYS AS IDENTITY (START WITH 10) PRIMARY KEY
```

O sistema vai cadastrar produtos e vendas: ninguém deve escolher o próximo código. O `START WITH` faz o contador começar onde o Capítulo 4 começava: produtos de 10 a 16, pedidos de 100 a 107.

O MENU • 1 a 5: REQUISITO 1 • 11 E 12: REQUISITO 2

```
    LOJA CONECTA

    MENU:

    1. Ver Produtos
    2. Ver Detalhes do Produto
    3. Cadastrar Produto
    4. Alterar Produto
    5. Apagar Produto

    11. Nova Venda
    12. Ver Venda

    0. Sair
```

verProdutos: LEFT JOIN com as categorias

```python
def verCategorias():
    print("CATEGORIAS")
    for cod, nome in banco.consultar("SELECT * FROM categorias ORDER BY cod_categoria;"):
        print(f"{cod} | {nome}")


def verProdutos():
    print("LISTA DE PRODUTOS")

    produtos = banco.consultar('''
        SELECT p.cod_produto, p.nome_produto, p.preco, p.estoque, c.nome_categoria
        FROM produtos p
        LEFT JOIN categorias c ON c.cod_categoria = p.cod_categoria
        ORDER BY p.cod_produto;
    ''')

    if (len(produtos) == 0):
        print("NÃO HÁ PRODUTOS CADASTRADOS")
        return

    print("Código | Produto | Preço | Estoque | Categoria")
    for cod, nome, preco, estoque, categoria in produtos:
        # LEFT JOIN: produto sem categoria chega com None
        if (categoria is None):
            categoria = "sem categoria"
        print(f"{cod} | {nome} | R$ {preco:,.2f} | {estoque} | {categoria}")
```

OPÇÃO 1

```
LISTA DE PRODUTOS
Código | Produto | Preço | Estoque | Categoria
10 | Notebook Lume 14 | R$ 3,499.00 | 5 |
Informática
...
15 | Mouse sem fio | R$ 89.90 | 30 | Acessórios
16 | Cabo HDMI | R$ 39.90 | 50 | sem categoria
```

- Por que **LEFT** JOIN? O Cabo HDMI não tem categoria: no INNER, ele sumiria da lista (Capítulo 4).
- A categoria dele chega como `None`, e o programa escreve *sem categoria*.
- `verCategorias` vai ajudar no cadastro.

escolherProduto: a ficha de um produto

```python
def escolherProduto():
    verProdutos()

    cod_escolhido = int(input("Digite o código do produto desejado: "))

    consulta = banco.consultar('''
        SELECT p.cod_produto, p.nome_produto, p.preco, p.estoque, p.cod_categoria, c.nome_categoria
        FROM produtos p
        LEFT JOIN categorias c ON c.cod_categoria = p.cod_categoria
        WHERE p.cod_produto = %s;
    ''', (cod_escolhido,))

    if (len(consulta) == 0):
        print("Produto não encontrado!")
        return None

    produto = consulta[0]

    print(f'''
    Ficha do Produto:

    Código: {produto[0]}
    Nome: {produto[1]}
    Preço: R$ {produto[2]:,.2f}
    Estoque: {produto[3]}
    Categoria: {produto[5] if produto[5] else "sem categoria"}
    ''')

    return produto
```

OPÇÃO 2

```
Digite o código do produto desejado: 16

    Ficha do Produto:

    Código: 16
    Nome: Cabo HDMI
    Preço: R$ 39.90
    Estoque: 50
    Categoria: sem categoria
```

- O molde do `escolherFuncionario`: mostra a lista, pede o código, consulta com `%s`.
- Devolve a linha do produto, ou `None`: o alterar e o apagar vão usar a mesma função.

cadastrarProduto: categoria opcional

```python
def cadastrarProduto():
    print("CADASTRO DE NOVO PRODUTO")

    nome = input("Digite o nome do produto: ")
    preco = float(input("Digite o preço: "))
    estoque = int(input("Digite o estoque: "))

    verCategorias()
    categoria = input("Digite o código da categoria (vazio = sem categoria): ").strip()

    # Vazio vira None, e o banco grava NULL: a FK aceita
    cod_categoria = None
    if (categoria):
        cod_categoria = int(categoria)

    try:
        banco.manipular('''
            INSERT INTO produtos
            VALUES (default, %s, %s, %s, %s);
        ''', (nome, preco, estoque, cod_categoria))
        print("PRODUTO CADASTRADO COM SUCESSO!")
    except Exception as error:
        print("ERRO AO CADASTRAR PRODUTO")
```

OPÇÃO 3

```
CADASTRO DE NOVO PRODUTO
Digite o nome do produto: Teclado mecânico
Digite o preço: 249.90
Digite o estoque: 15
CATEGORIAS
...
Digite o código da categoria (vazio = sem categoria): 3
PRODUTO CADASTRADO COM SUCESSO!
```

- Categoria vazia vira `None`, e o banco grava `NULL`: a FK aceita.
- `VALUES (default, …)` na ordem do CREATE: nome, preço, estoque, categoria.
- Na opção 1, o teclado aparece como **17**.

alterarProduto: deixe vazio para manter

```
    # ... escolherProduto(), a confirmação S/N e o desempacotamento,
    #     como no alterarFuncionario
    cod, nome, preco, estoque, cod_categoria, nome_categoria = produto
    print("ALTERAR DADOS:")
    print("(Deixe vazio para manter o original)")
    novo_nome = input(f"Digite o novo nome ({nome}): ").strip()
    if (novo_nome):
        nome = novo_nome
    novo_preco = input(f"Digite o novo preço (R$ {preco:,.2f}): ").strip()
    if (novo_preco):
        preco = float(novo_preco)
    novo_estoque = input(f"Digite o novo estoque ({estoque}): ").strip()
    if (novo_estoque):
        estoque = int(novo_estoque)
    verCategorias()
    nova_categoria = input(f"Digite o código da nova categoria ({cod_categoria}): ").strip()
    if (nova_categoria):
        cod_categoria = int(nova_categoria)
    try:
        banco.manipular('''
            UPDATE produtos
            SET nome_produto = %s,
                preco = %s,
                estoque = %s,
                cod_categoria = %s
            WHERE cod_produto = %s;
        ''', (nome, preco, estoque, cod_categoria, cod))
        print("PRODUTO ATUALIZADO!")
    except Exception as error:
        print("Ocorreu um erro na atualização!")
```

OPÇÃO 4 • PRODUTO 17

```
Deseja alterar as informações do produto? (S/N) S
ALTERAR DADOS:
(Deixe vazio para manter o original)
Digite o novo nome (Teclado mecânico):
Digite o novo preço (R$ 249.90): 229.90
Digite o novo estoque (15):
...
Digite o código da nova categoria (3):
PRODUTO ATUALIZADO!
```

- O mesmo caminho do `alterarFuncionario`: escolher, confirmar, desempacotar, substituir o que veio digitado.
- O código do produto vai no último `%s`, o do WHERE.

removerProduto: produto vendido fica

```python
def removerProduto():
    print("REMOÇÃO DE PRODUTO")

    produto = escolherProduto()

    if (not produto):
        print("Produto não encontrado!")
        return

    continuar = input("DESEJA REALMENTE APAGAR ESSE PRODUTO? (S/N) ")

    if (continuar != "S"):
        print("Cancelando operação...")
        return

    try:
        banco.manipular('''
            DELETE FROM produtos
            WHERE cod_produto = %s;
        ''', (produto[0],))
        print("Produto removido com sucesso!")
    except Exception as error:
        # Produto que já foi vendido não sai: a FK fk_item_produto recusa
        print("Não foi possível remover o produto! Ele já foi vendido?")
```

OPÇÃO 5 · PRODUTO 16

```
(a ficha do produto 16, Cabo HDMI)
DESEJA REALMENTE APAGAR ESSE PRODUTO? (S/N) S
ERRO DE MANIPULAÇÃO: update or delete on table "produtos"
violates foreign key constraint "fk_item_produto" on table
"itens_pedido"
Não foi possível remover o produto! Ele já foi vendido?
```

OPÇÃO 5 · PRODUTO 15

```
(a ficha do produto 15, Mouse sem fio)
DESEJA REALMENTE APAGAR ESSE PRODUTO? (S/N) S
Produto removido com sucesso!
```

O Cabo HDMI está num pedido: a FK `fk_item_produto` do Capítulo 4 recusa o DELETE.

REQUISITO 1 • TESTES

O CRUD de produtos funcionando

TESTE, LOGO DEPOIS DO SETUP	ESPERADO
1 • Ver produtos	Os produtos de 10 a 16; o Cabo HDMI com *sem categoria*
2 • produto 16	A ficha do Cabo HDMI, categoria *sem categoria*
3 • Teclado mecânico • 249.90 • 15 • categoria 3	*PRODUTO CADASTRADO COM SUCESSO!*; ele aparece como 17
3 • Headset • 199 • 10 • categoria 9	A classe mostra *violates foreign key constraint "fk_prod_categoria"*; o programa, *ERRO AO CADASTRAR PRODUTO*
3 • Headset • –5 • 10 • sem categoria	A classe mostra *violates check constraint "ck_preco"*; nada entra
4 • produto 17 • S • preço 229.90	*PRODUTO ATUALIZADO!*; o resto continua igual
5 • produto 16 • S	*Não foi possível remover o produto! Ele já foi vendido?*
5 • produto 15 • S	*Produto removido com sucesso!*

PENSE E RESPONDA O Mouse sem fio (15) saiu e o Cabo HDMI (16) não. Em qual pedido do Capítulo 4 está o cabo?

REQUISITO 2 • FLUXO DE VENDA • A IDEIA

Uma venda: primeiro o pedido, depois os itens

1 · Escolher o cliente: a lista e o código digitado

↓

2 · Montar os itens no Python: uma lista de tuplas **(produto, qtd, preço)**. Nada vai ao banco ainda.

↓ registrarVenda(cod_cliente, itens)

3 · INSERT INTO pedidos … RETURNING cod_pedido: o banco gera o **108** e devolve

4 · INSERT INTO itens_pedido, um para cada item, com o 108

↓

commit(): o pedido e os itens, juntos

algum INSERT falhou? **rollback()**: nem o pedido fica

- Cada item precisa do `cod_pedido`, que o banco ainda vai gerar. O `RETURNING` (Capítulos 3 e 4) faz o INSERT responder com esse código.
- Por que não usar o `banco.manipular`? Ele abre, grava e fecha a cada comando. Se o 2º item falhasse, o pedido já estaria gravado, pela metade.
- Por isso, um **método novo** na classe: `registrarVenda` usa **uma conexão só** para o pedido e todos os itens, e um `commit()` só, no fim.

Isso é uma **transação**: vários comandos que o banco trata como um só. O tudo ou nada da Seção 01, agora com uma razão de negócio.

classBanco.py: registrarVenda

```
    def registrarVenda(self, cod_cliente, itens):
        conexao = None
        try:
            conexao = self.conectar()
            cursor = conexao.cursor()
            # 1. O pedido. O RETURNING devolve o código que o banco gerou
            cursor.execute('''
                INSERT INTO pedidos
                VALUES (default, %s, default, default)
                RETURNING cod_pedido;
            ''', (cod_cliente,))
            cod_pedido = cursor.fetchone()[0]
            # 2. Os itens, todos com o código do pedido que acabou de nascer
            for cod_produto, qtd, preco_unit in itens:
                cursor.execute('''
                    INSERT INTO itens_pedido
                    VALUES (default, %s, %s, %s, %s);
                ''', (cod_pedido, cod_produto, qtd, preco_unit))
            # 3. Tudo certo: o pedido e os itens são gravados juntos
            conexao.commit()
            return cod_pedido
        except Exception as error:
            print("ERRO NA VENDA:", error)
            # Algum passo falhou: nem o pedido nem os itens ficam gravados
            if (conexao is not None):
                conexao.rollback()
            raise
        finally:
            if (conexao is not None):
                conexao.close()
```

- Acrescente o método **dentro** da classe, depois do `manipular`.
- `VALUES (default, %s, default, default)`: o código, a data e o status vêm do banco; só o cliente é informado.
- `fetchone()` pega a linha que o RETURNING devolveu; o `[0]`, o código.
- O `for` grava cada item com esse código.
- Um `commit()` só, depois do `for`. No erro, o `rollback()` desfaz até o pedido, e o `raise` avisa quem chamou.

cadastrarVenda: cliente, itens e o método

```python
def cadastrarVenda():
    print("NOVA VENDA")
    # 1. O cliente
    verClientes()
    cod_cliente = int(input("Digite o código do cliente: "))
    # 2. Os itens: montados numa lista, antes de gravar qualquer coisa
    itens = []
    verProdutos()
    while (True):
        cod_produto = int(input("Código do produto (0 para fechar a venda): "))
        if (cod_produto == 0):
            break
        consulta = banco.consultar('''
            SELECT cod_produto, nome_produto, preco FROM produtos
            WHERE cod_produto = %s;
        ''', (cod_produto,))
        if (len(consulta) == 0):
            print("Produto não encontrado!")
            continue
        cod, nome, preco = consulta[0]
        qtd = int(input(f"Quantidade de {nome}: "))
        # O preço do dia vai junto: é o preco_unit do item
        itens.append((cod, qtd, preco))
        print(f"{qtd} x {nome} na venda.")
    if (len(itens) == 0):
        print("Venda sem itens. Nada foi gravado.")
        return
    # 3. Gravar: o pedido e os itens juntos, pelo método da classe
    try:
        cod_pedido = banco.registrarVenda(cod_cliente, itens)
        print(f"VENDA {cod_pedido} REGISTRADA COM {len(itens)} ITEM(NS)!")
    except Exception as error:
        print("Não foi possível registrar a venda. Nada foi gravado.")
```

OPÇÃO 11

```
NOVA VENDA
(a lista de clientes)
Digite o código do cliente: 6
(a lista de produtos)
Código do produto (0 para fechar a venda): 13
Quantidade de Fone Bluetooth: 2
2 x Fone Bluetooth na venda.
Código do produto (0 para fechar a venda): 17
Quantidade de Teclado mecânico: 1
1 x Teclado mecânico na venda.
Código do produto (0 para fechar a venda): 0
VENDA 108 REGISTRADA COM 2 ITEM(NS)!
```

O preço vem do cadastro e vai para o `preco_unit`: o preço do dia da venda, mesmo que o produto mude depois.

verVenda: os itens e o total

```python
def verVenda():
    print("DETALHES DA VENDA")
    cod_pedido = int(input("Digite o código da venda: "))
    consulta = banco.consultar('''
        SELECT p.cod_pedido, c.nome_cliente, p.data_pedido, p.status
        FROM pedidos p
        JOIN clientes c ON c.cod_cliente = p.cod_cliente
        WHERE p.cod_pedido = %s;
    ''', (cod_pedido,))
    if (len(consulta) == 0):
        print("Venda não encontrada!")
        return
    cod, cliente, data, status = consulta[0]
    print(f"Venda {cod} | {cliente} | {data:%d/%m/%Y} | {status}")
    itens = banco.consultar('''
        SELECT pr.nome_produto, i.qtd, i.preco_unit, i.qtd * i.preco_unit
        FROM itens_pedido i
        JOIN produtos pr ON pr.cod_produto = i.cod_produto
        WHERE i.cod_pedido = %s
        ORDER BY i.cod_item;
    ''', (cod_pedido,))
    total = 0
    for nome, qtd, preco_unit, subtotal in itens:
        print(f"  {qtd} x {nome} (R$ {preco_unit:,.2f}) = R$ {subtotal:,.2f}")
        total = total + subtotal
    if (len(itens) == 0):
        print("  (sem itens)")
    print(f"TOTAL: R$ {total:,.2f}")
```

OPÇÃO 12

```
Digite o código da venda: 108
Venda 108 | Gustavo Ramos | 13/11/2026 | aberto
  2 x Fone Bluetooth (R$ 199.90) = R$ 399.80
  1 x Teclado mecânico (R$ 229.90) = R$ 229.90
TOTAL: R$ 629.70
```

OPÇÃO 12

```
Digite o código da venda: 107
Venda 107 | Mariana Costa | 05/10/2026 | aberto
  (sem itens)
TOTAL: R$ 0.00
```

Duas consultas: o cabeçalho (com JOIN no cliente) e os itens. O subtotal sai do SQL; o total, do `for` no Python. A data é a do dia da venda (`DEFAULT CURRENT_DATE`).

REQUISITO 2 • TESTES

Uma venda que passa e três que voltam atrás

OPÇÃO 11 • NA ORDEM	ESPERADO
Cliente 6 • 2 × Fone (13) • 1 × Teclado (17)	**VENDA 108 REGISTRADA COM 2 ITEM(NS)!**
Cliente 99 • 1 × Fone (13)	*ERRO NA VENDA: … fk_ped_cliente • Não foi possível registrar a venda. Nada foi gravado.*
Cliente 2 • 1 × Fone (13) • 2 × Fone (13)	*ERRO NA VENDA: duplicate key … "uq_item_pedido_produto" • DETAIL: Key (cod_pedido, cod_produto)= (110, 13)*
Cliente 2 • 0 × Smartphone (12)	*ERRO NA VENDA: … violates check constraint "ck_qtd"*
Cliente 2 • produto 99 • depois 0	*Produto não encontrado! • Venda sem itens. Nada foi gravado.*
Cliente 2 • 1 × Capa para celular (14)	**VENDA 112 REGISTRADA COM 1 ITEM(NS)!**

DEU CERTO SE…

No pgAdmin, existem as vendas 108 e 112, e nenhuma entre 109 e 111. A tabela `itens_pedido` tem 14 itens: os 11 do setup, os 2 da venda 108 e o da 112. Nenhum item órfão.

PENSE E RESPONDA

No teste do Fone repetido, o DETAIL fala do pedido 110: ele chegou a existir durante a venda. Por que não está no banco? E por que a venda seguinte foi a 112?

PRÁTICA GUIADA • PARA IR ALÉM

Desafios da Loja Conecta

1 Baixar o estoque na venda

No `registrarVenda`, depois de cada item: `UPDATE produtos SET estoque = estoque - %s`. Venda maior que o estoque? O `ck_estoque` recusa, e o rollback desfaz a venda inteira.

2 Vendas de um cliente

Opção 13: escolha o cliente e liste as vendas dele, com data e status.

3 Cancelar uma venda

Mude o status para `cancelado`. Valor fora da lista? O `ck_status` do Capítulo 4 recusa.

4 CRUD de clientes

O mesmo caminho dos produtos, para os clientes. E-mail repetido? O `UNIQUE` recusa, e o `except` avisa.

O gabarito dos dois requisitos está no repositório: `Material/Capitulo 5 - Python e PostgreSQL/loja`. Compare depois de fazer, não antes.

ATIVIDADE FINAL • INDIVIDUAL • TEMA SORTEADO

Do briefing ao sistema

Cada um sorteia um tema e constrói o próprio banco e o próprio sistema: da modelagem ao CRUD em Python.

1 Modelar
DER e MER no draw.io, a partir do briefing

→

2 Banco
o setup.py na Aiven: tabelas, regras e dados

→

3 CRUD
classBanco.py e o menu de um cadastro

→

4 Movimento
o registro com itens, RETURNING e transação

Usa os Capítulos 1 a 5: entidades, cardinalidade, DER, MER, formas normais, restrições, scripts, JOIN, .env, psycopg, parâmetros e transação.

ATIVIDADE FINAL • O QUE ENTREGAR

Um sistema completo, no seu tema

1 • MODELAGEM

- Entidades, relacionamentos e cardinalidade do briefing.
- DER em Chen, sem atributos.
- MER com todas as tabelas, PK e FK, ligadas com 1 e N.
- O N:N do movimento vira tabela de itens.

2 • BANCO • SETUP.PY

- DROP, CREATE e INSERT no molde do `setup_rh.py`. Roda duas vezes seguidas.
- Toda PK com `GENERATED ALWAYS`.
- Toda FK, CHECK e UNIQUE com nome: uma regra para cada exigência do briefing.
- Dados para testar: 5 linhas em cada cadastro e 3 movimentos com itens.

3 E 4 • SISTEMA • MAIN.PY

- `classBanco.py` com `consultar`, `manipular` e um método para o movimento, como o `registrarVenda`.
- CRUD completo de um cadastro com FK: ver, ver detalhes, cadastrar, alterar e apagar.
- O movimento: o cabeçalho com RETURNING e os itens, numa transação só.
- Ver um movimento com os itens e o total.
- Nenhuma opção do menu termina em Traceback.

Entrega: repositório no GitHub com o DER/MER (imagem ou `.drawio`), o `.env.exemplo`, o `.gitignore` e os `.py`. O `.env` **não** vai: confira no `git status`.

ATIVIDADE FINAL • TEMAS PARA O SORTEIO • 1 A 6

Os temas

1 Pizzaria Forno Bom

Os sabores (nome, preço) de cada categoria: tradicional, especial ou doce. Os clientes têm telefone único e endereço. Cada pedido é de um cliente e tem vários sabores, cada um com tamanho (P, M ou G) e quantidade.

Movimento: pedido com os sabores

2 Farmácia Bem-Estar

Os remédios (nome, preço, estoque, se exige receita) de cada laboratório. Os clientes têm CPF único. Cada venda é de um cliente e tem vários remédios com quantidade. O estoque nunca fica negativo.

Movimento: venda com os remédios

3 Pousada Mar Azul

Os quartos (número único, andar) de cada tipo, com o valor da diária. Os hóspedes têm CPF único. Cada reserva é de um hóspede, com entrada e saída (a saída depois da entrada), e pode incluir vários quartos.

Movimento: reserva com os quartos

4 Pet Feliz • Banho e Tosa

Os tutores (nome, telefone único) e os pets de cada um (nome, espécie: cão ou gato, porte P, M ou G). Os serviços têm preço. Cada agendamento é de um pet, com data e horário, e inclui vários serviços.

Movimento: agendamento com os serviços

5 Salão Beleza Pura

Os serviços (nome, duração em minutos, preço) de cada categoria: cabelo, unhas ou estética. Os clientes têm telefone único. Cada atendimento é de um cliente, numa data, e inclui vários serviços. Duração e preço maiores que zero.

Movimento: atendimento com os serviços

6 Cantina da Escola

Os produtos (nome, preço, estoque) de cada categoria: salgados, bebidas e doces. Os alunos têm matrícula única e turma. Cada pedido é de um aluno e tem vários produtos com quantidade. O estoque nunca fica negativo.

Movimento: pedido com os produtos

ATIVIDADE FINAL • TEMAS PARA O SORTEIO • 7 A 12

Os temas

7 Sebo Página Dois

Os livros (título, autor, preço, quantidade, estado: bom, regular ou ruim) de cada gênero. Os clientes têm e-mail único. Cada venda é de um cliente e tem vários livros com quantidade. O estoque nunca fica negativo.

Movimento: venda com os livros

8 Restaurante Sabor da Casa

Os pratos (nome, preço) de cada categoria: entrada, principal ou sobremesa. As mesas têm número único e de 1 a 12 lugares. Cada comanda é de uma mesa, numa data, está aberta ou fechada e tem vários pratos com quantidade.

Movimento: comanda com os pratos

9 Locadora Rota Livre

Os carros (placa única, modelo, ano) de cada categoria, com o valor da diária. Os clientes têm CNH única. Cada locação é de um cliente, com retirada e devolução (a devolução depois da retirada), e pode incluir vários carros.

Movimento: locação com os carros

10 Floricultura Jardim

Os arranjos (nome, preço, estoque) de cada tipo: buquê, cesta ou vaso. Os clientes têm telefone único. Cada encomenda é de um cliente, com endereço e data de entrega (nunca antes do dia da encomenda), e tem vários arranjos.

Movimento: encomenda com os arranjos

11 Lava-jato Brilho Total

Os clientes (nome, telefone único) e os veículos de cada um (placa única, modelo, tipo: carro, moto ou caminhonete). Os serviços têm preço. Cada atendimento é de um veículo, numa data, e inclui vários serviços.

Movimento: atendimento com os serviços

12 Agência Viaje Bem

Os pacotes (destino, data de saída, preço, vagas) de cada tipo: praia, serra ou cidade. Os clientes têm CPF único. Cada reserva é de um cliente e inclui um ou mais pacotes, com o número de pessoas. As vagas nunca ficam negativas.

Movimento: reserva com os pacotes

Os temas

13 Doceria Doce Encontro

Os doces e bolos (nome, preço, estoque) de cada categoria. Os clientes têm telefone único. Cada encomenda é de um cliente, com data de retirada, e tem vários itens com quantidade maior que zero. O estoque nunca fica negativo.

Movimento: encomenda com os doces

14 Água & Gás Express

Os produtos (galão de água, botijão de gás: preço e estoque). Os bairros atendidos têm taxa de entrega. Os clientes moram num bairro e têm telefone único. Cada pedido é de um cliente e tem vários produtos com quantidade.

Movimento: pedido com os produtos

15 Brechó Segunda Chance

As peças (descrição, tamanho de PP a GG, preço, estado: novo, seminovo ou usado) de cada categoria. Os clientes têm e-mail único. Cada venda é de um cliente e tem várias peças. Cada peça é única: só pode ser vendida uma vez.

Movimento: venda com as peças

16 Ótica Visão Clara

Os produtos (armação, lente ou óculos de sol: preço e estoque) de cada marca. Os clientes têm CPF único. Cada venda é de um cliente e tem vários produtos com quantidade. O estoque nunca fica negativo.

Movimento: venda com os produtos

17 Locadora de Jogos Joystick

Os jogos (título, preço do aluguel, cópias disponíveis) de cada plataforma. Os clientes têm e-mail único. Cada aluguel é de um cliente, com a devolução prevista, e inclui vários jogos. As cópias nunca ficam negativas.

Movimento: aluguel com os jogos

18 Hortifruti Feira Verde

Os produtos (nome, preço por kg, estoque em kg) de cada seção: frutas, verduras ou legumes. Os clientes têm telefone único. Cada venda é de um cliente e tem vários produtos, cada um com o peso em kg, maior que zero e com casas decimais.

Movimento: venda com os produtos

ATIVIDADE FINAL • ANTES DE ENTREGAR

Confira o seu sistema

MODELAGEM

- [ ] Toda entidade do briefing virou tabela.
- [ ] Toda FK fica no lado N.
- [ ] O movimento tem a tabela de itens, com as duas FKs.

BANCO

- [ ] `setup.py` roda duas vezes seguidas, sem erro.
- [ ] Toda PK com GENERATED ALWAYS.
- [ ] Cada exigência do briefing tem uma regra com nome.
- [ ] Testei um dado errado para cada regra.

SISTEMA

- [ ] O `.env` não aparece no `git status`.
- [ ] Todo valor digitado entra por `%s`.
- [ ] Movimento com erro não deixa nada pela metade.
- [ ] Nenhuma opção do menu termina em Traceback.

Sugestão: comece pela modelagem e mostre ao professor antes de escrever o `setup.py`. Um MER certo poupa horas de ALTER TABLE.

CAPÍTULO 5 • CHECKLIST

Confira antes de chamar o professor

CONEXÃO E SETUP

- [ ] `.env` com os 6 dados da Aiven, na pasta do programa.
- [ ] `.gitignore` com `.env`, e o `.env.exemplo` sem senha.
- [ ] `setup_rh.py` e `setup_cinema.py` rodam duas vezes seguidas.
- [ ] O `finally` fecha o cursor e a conexão.

PRIMEIRO SISTEMA E CLASSE

- [ ] `sistema_cinema.py` com o menu e os cadastros (Prática 2).
- [ ] Todo valor digitado com `%s`, sem aspas; um valor só com `(valor,)`.
- [ ] Nenhuma f-string montando SQL.
- [ ] `classBanco.py` com `consultar` e `manipular`, `except` e `raise`.

CRUD E LOJA

- [ ] CRUD do cinema completo (Prática 3).
- [ ] CRUD de produtos da loja (requisito 1).
- [ ] Venda gravada inteira ou nada, pelo `registrarVenda` (requisito 2).
- [ ] Os testes da loja conferidos no pgAdmin.

Resumo

.env
NOME=valor, um por linha
fica fora do GitHub
o .env.exemplo vai

Ler o .env
load_dotenv()
os.getenv("DB_HOST")
texto, ou None

Conectar
psycopg.connect(host,
port, dbname, user,
password, sslmode)

Cursor
execute(sql, parametros)
fetchall() · fetchone()
rowcount

Transação
commit() grava
rollback() desfaz
close() sem commit desfaz

try • except • finally
except Exception: mostra
rollback, se conectou
finally fecha tudo

Parâmetros
%s sem aspas
(valor,) com vírgula
VALUES (default, %s…)

Tipos
numeric → Decimal
date → datetime.date
NULL ↔ None

Classe Banco
consultar → linhas
manipular → rowcount
except mostra · raise

CRUD
ver · escolher · cadastrar
alterar: vazio mantém
remover: confirma S/N

RETURNING
INSERT ... RETURNING cod
cursor.fetchone()[0]
o código gerado

Venda
registrarVenda: pedido
e itens, 1 conexão
commit no fim

Armadilhas: .env salvo como .env.txt • senha no código • esquecer o commit • (cod) sem vírgula • '%s' com aspas • f-string no SQL • except sem raise na classe • cursor e conexão fora do finally • manipular para cada parte da venda

BANCO DE DADOS · CAPÍTULO CONCLUÍDO

Seu programa fala com o banco!

Sorteie o seu tema e mãos à obra. Dúvidas? Discord aberto.

DISCORD
discord.gg/wt5CVZZWJs

DRIVE
drive.google.com/drive/folders/1hOl0DaPeAor7gnhKBUIlZ5n8lLRDNvUY

GITHUB
github.com/TarikPonciano/Programador-de-Sistema-SENAC

psycopg.org/psycopg3/docs — o guia do psycopg

pypi.org/project/python-dotenv — o .env

console.aiven.io — o seu serviço